

Tugas 10: Praktikum Mandiri 10 Machine Learning

Hisyam Wildan Alfath - 0110222206

Teknik Informatika, STT Terpadu Nurul Fikri, Depok

E-mail: hisy22206ti@student.nurulfikri.ac.id

Abstract. Praktikum mandiri ini bertujuan untuk membangun dan mengevaluasi model klasifikasi tipe cuaca menggunakan algoritma K-Nearest Neighbors (KNN). Dataset cuaca diproses melalui tahap preprocessing, meliputi pemisahan fitur numerik dan kategorikal, standardisasi menggunakan StandardScaler, serta transformasi kategorikal dengan OneHotEncoder. Selanjutnya, model KNN dibangun dalam bentuk pipeline agar preprocessing dan pelatihan berjalan otomatis dan terstruktur. Hasil evaluasi dengan accuracy, classification report, dan confusion matrix menunjukkan model mampu melakukan prediksi dengan performa yang baik, sehingga KNN dinilai efektif untuk klasifikasi cuaca pada data yang telah dipersiapkan dengan tepat.

1. Praktikum Mandiri 10

Latihan 1 - KNN Sederhana (Prediksi Panas/Dingin)

- Import Library untuk KNN dan Pengolahan Data

```
[1]
✓ 3s      import pandas as pd
          import numpy as np
          import matplotlib.pyplot as plt

          from sklearn.neighbors import KNeighborsClassifier
          from sklearn.model_selection import LeaveOneOut, cross_val_score
          from sklearn.preprocessing import StandardScaler
```

Kode ini berfungsi memanggil pustaka seperti pandas, numpy, matplotlib, StandardScaler, KNeighborsClassifier, dan LeaveOneOut. Input-nya adalah perintah import dari masing-masing library. Output-nya tidak menampilkan apa pun di layar, tetapi library tersebut siap dipakai untuk tahap berikutnya seperti membuat data, menormalisasi fitur, dan membangun model KNN.

- Membuat Data Training dalam DataFrame

```
[2] ✓ 0s
data_train = {
    "Temperatur": [10, 25, 15, 20, 18, 20, 22, 24],
    "Angin": [0, 0, 5, 3, 7, 10, 5, 6],
    "Label": ["Dingin", "Panas", "Dingin", "Panas", "Dingin", "Dingin", "Panas", "Panas"]
}

df = pd.DataFrame(data_train)
df
```

	Temperatur	Angin	Label
0	10	0	Dingin
1	25	0	Panas
2	15	5	Dingin
3	20	3	Panas
4	18	7	Dingin
5	20	10	Dingin
6	22	5	Panas
7	24	6	Panas

Kode memasukkan data cuaca (Temperatur, Angin, Label) ke dalam dictionary lalu mengubahnya menjadi DataFrame df. Input-nya adalah nilai-nilai cuaca dan label kategori “Dingin” atau “Panas”. Output-nya berupa tabel DataFrame berisi 8 baris data training yang akan dipakai sebagai contoh pola untuk model.

- Mengubah Label Teks Menjadi Angka (Encoding)

```
[3] ✓ 0s
df["Label_num"] = df["Label"].map({"Dingin": 0, "Panas": 1})
df
```

	Temperatur	Angin	Label	Label_num
0	10	0	Dingin	0
1	25	0	Panas	1
2	15	5	Dingin	0
3	20	3	Panas	1
4	18	7	Dingin	0
5	20	10	Dingin	0
6	22	5	Panas	1
7	24	6	Panas	1

Kode melakukan mapping label dari teks ke numerik, yaitu “Dingin” menjadi 0 dan “Panas” menjadi 1. Input-nya adalah kolom Label pada DataFrame. Output-nya berupa kolom baru Label_num yang berisi angka sehingga model KNN bisa memproses targetnya secara matematis.

- Memisahkan Fitur (X) dan Target (y)

```
[4]
✓ 0s      X = df[["Temperatur", "Angin"]]
          y = df["Label_num"]
```

Kode memisahkan kolom fitur ke variabel X (Temperatur dan Angin) serta target ke variabel y (Label_num). Input-nya adalah DataFrame yang sudah memiliki label numerik. Output-nya adalah dua variabel: X berisi data input model dan y berisi jawaban yang ingin dipelajari model.

- Normalisasi Fitur dengan StandardScaler

```
[5]
✓ 0s      scaler = StandardScaler()
          X_scaled = scaler.fit_transform(X)

          X_scaled

array([[ -2.00956415, -1.40556386],
       [  1.24918853, -1.40556386],
       [-0.92331326,  0.15617376],
       [  0.16293763, -0.46852129],
       [-0.27156272,  0.78086881],
       [  0.16293763,  1.71791138],
       [  0.59743799,  0.15617376],
       [  1.03193835,  0.46852129]])
```

Kode menggunakan StandardScaler() untuk menstandarkan skala Temperatur dan Angin. Input-nya adalah data fitur X, lalu diproses dengan fit_transform. Output-nya berupa X_scaled yang nilainya sudah berada pada skala seragam, sehingga perhitungan jarak KNN lebih akurat.

- Memprediksi Kondisi Cuaca dengan KNN (k=3)

```
[6]
✓ 0s      test_point = np.array([[16, 3]]) # data test asli
          test_scaled = scaler.transform(test_point) # ikut scaling

          knn_3 = KNeighborsClassifier(n_neighbors=3)
          knn_3.fit(X_scaled, y)

          pred = knn_3.predict(test_scaled)[0]
          label_pred = "Panas" if pred==1 else "Dingin"

          label_pred

/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but StandardScaler was fitted with feature names
  warnings.warn(
'Dingin'
```

Kode memberi titik uji [[16,3]] lalu menskalakannya dan diprediksi oleh model KNN dengan jumlah tetangga k=3. Input-nya adalah data training yang sudah dinormalisasi dan satu data test. Output-nya berupa hasil prediksi kelas, yaitu titik tersebut diperkirakan masuk kategori “Dingin”.

Latihan 2 - Confusion Matrix & Metrik Evaluasi

- Import Library untuk Evaluasi Klasifikasi

```
[7] import pandas as pd
✓ 0s import numpy as np

from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score, classification_report
```

Kode memanggil fungsi evaluasi seperti `confusion_matrix`, `accuracy_score`, `precision_score`, `recall_score`, dan `classification_report`. Input-nya adalah perintah `import`. Output-nya tidak ada tampilan langsung, tapi semua fungsi evaluasi siap digunakan untuk menghitung performa model.

- Membuat Data Aktual dan Prediksi ke DataFrame

```
[8] data = {
✓ 0s   "NIM": ["TI001", "TI002", "TI003", "TI004", "TI005", "TI006", "TI007", "TI008", "TI009", "TI010"],
      "Aktual": ["Lulus", "Lulus", "Lulus", "Lulus", "Lulus",
                "Tidak Lulus", "Tidak Lulus", "Tidak Lulus", "Tidak Lulus"],
      "Prediksi": ["Lulus", "Lulus", "Lulus", "Tidak Lulus", "Tidak Lulus",
                   "Lulus", "Tidak Lulus", "Tidak Lulus", "Tidak Lulus", "Tidak Lulus"]}
df = pd.DataFrame(data)
df
```

	NIM	Aktual	Prediksi
0	TI001	Lulus	Lulus
1	TI002	Lulus	Lulus
2	TI003	Lulus	Lulus
3	TI004	Lulus	Tidak Lulus
4	TI005	Lulus	Tidak Lulus
5	TI006	Tidak Lulus	Lulus
6	TI007	Tidak Lulus	Tidak Lulus
7	TI008	Tidak Lulus	Tidak Lulus
8	TI009	Tidak Lulus	Tidak Lulus
9	TI010	Tidak Lulus	Tidak Lulus

Kode menuliskan data NIM, label aktual, dan label prediksi ke dictionary lalu menjadi DataFrame `df`. Input-nya berupa daftar status “Lulus/Tidak Lulus” aktual dan prediksi. Output-nya DataFrame 10 baris yang memperlihatkan perbandingan hasil sebenarnya dan hasil model.

- Encoding Label Aktual dan Prediksi Menjadi Numerik

```
[9]
✓ Os
df["Aktual_num"] = df["Aktual"].map({"Tidak Lulus":0, "Lulus":1})
df["Prediksi_num"] = df["Prediksi"].map({"Tidak Lulus":0, "Lulus":1})

df
```

	NIM	Aktual	Prediksi	Aktual_num	Prediksi_num
0	TI001	Lulus	Lulus	1	1
1	TI002	Lulus	Lulus	1	1
2	TI003	Lulus	Lulus	1	1
3	TI004	Lulus	Tidak Lulus	1	0
4	TI005	Lulus	Tidak Lulus	1	0
5	TI006	Tidak Lulus	Lulus	0	1
6	TI007	Tidak Lulus	Tidak Lulus	0	0
7	TI008	Tidak Lulus	Tidak Lulus	0	0
8	TI009	Tidak Lulus	Tidak Lulus	0	0
9	TI010	Tidak Lulus	Tidak Lulus	0	0

Kode mengubah label “Tidak Lulus” menjadi 0 dan “Lulus” menjadi 1 untuk kedua kolom. Input-nya adalah kolom Aktual dan Prediksi. Output-nya kolom Aktual_num dan Prediksi_num sehingga evaluasi bisa dihitung lebih mudah.

- Menentukan y_true dan y_pred untuk Evaluasi

```
[10]
✓ Os
y_true = df["Aktual_num"]
y_pred = df["Prediksi_num"]
```

Kode mengambil Aktual_num sebagai y_true dan Prediksi_num sebagai y_pred. Input-nya adalah DataFrame yang sudah memiliki label angka. Output-nya dua vektor angka yang akan dibandingkan untuk menghitung confusion matrix dan metrik lainnya.

- Menghasilkan Confusion Matrix

```
[11] ✓ 0s cm = confusion_matrix(y_true, y_pred)
cm

array([[4, 1],
       [2, 3]])

[12] ✓ 0s cm_df = pd.DataFrame(
    cm,
    index=["Aktual Tidak Lulus (0)", "Aktual Lulus (1)"],
    columns=["Prediksi Tidak Lulus (0)", "Prediksi Lulus (1)"]
)
cm_df
```

	Prediksi Tidak Lulus (0)	Prediksi Lulus (1)
Aktual Tidak Lulus (0)	4	1
Aktual Lulus (1)	2	3

Kode menjalankan `confusion_matrix(y_true, y_pred)`. Input-nya label aktual dan prediksi dalam bentuk numerik. Output-nya matriks `[[4,1],[2,3]]` yang menunjukkan jumlah prediksi benar dan salah untuk masing-masing kelas.

- Menghitung Accuracy, Precision, dan Recall

```
[13] ✓ 0s acc = accuracy_score(y_true, y_pred)
prec = precision_score(y_true, y_pred) # positif = Lulus (1)
rec = recall_score(y_true, y_pred)

acc, prec, rec

(0.7, 0.75, 0.6)

[14] ✓ 0s print("Accuracy :", acc*100, "%")
print("Precision :", prec*100, "%")
print("Recall    :", rec*100, "%")

Accuracy : 70.0 %
Precision : 75.0 %
Recall    : 60.0 %
```

Kode menghitung metrik evaluasi utama dari `y_true` dan `y_pred`. Input-nya adalah dua vektor label tadi. Output-nya nilai Accuracy 0.70, Precision 0.75, dan Recall 0.60, yang menjelaskan seberapa tepat dan seberapa lengkap prediksi model.

- Menampilkan Classification Report Lengkap

```
[15]
✓ 0s
print(classification_report(y_true, y_pred, target_names=["Tidak Lulus", "Lulus"]))
```

	precision	recall	f1-score	support
Tidak Lulus	0.67	0.80	0.73	5
Lulus	0.75	0.60	0.67	5
accuracy			0.70	10
macro avg	0.71	0.70	0.70	10
weighted avg	0.71	0.70	0.70	10

Kode mencetak classification_report untuk dua kelas. Input-nya label aktual, label prediksi, dan nama kelas. Output-nya tabel evaluasi berisi precision, recall, f1-score, dan support per kelas, plus akurasi total sebagai ringkasan kualitas model.

Latihan 3 - KNN Dataset Cuaca Besar + Pipeline

- Import Library untuk Olah Data, Preprocessing, dan KNN

```
[16]
✓ 1s
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, GridSearchCV, cross_val_score
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import confusion_matrix, classification_report, accuracy_score
```

Kode ini memanggil library seperti pandas, numpy, seaborn, train_test_split, ColumnTransformer, StandardScaler, OneHotEncoder, Pipeline, KNeighborsClassifier, serta metrics evaluasi. Input-nya adalah perintah import semua library tersebut. Output-nya tidak menampilkan hasil apa pun, tetapi semua tools siap dipakai untuk proses data dan training model.

- Mount Google Drive

```
[17]
✓ 33s
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

Kode melakukan drive.mount('/content/drive') agar file dataset di Google Drive bisa diakses. Input-nya adalah perintah mount dan izin akses drive. Output-nya berupa status bahwa drive sudah terhubung sehingga path file bisa dibaca.

- Membaca Dataset CSV dan Menampilkan Contoh Data Awal

```
[18] df = pd.read_csv("/content/drive/MyDrive/Hisyam Wildan Alfath/Semester 7/Machine Learning/Pertemuan 10/Praktikum dan Tugas Mandiri 10/data/weather_classification_data.csv")
df.head()
```

	Temperature	Humidity	Wind Speed	Precipitation (%)	Cloud Cover	Atmospheric Pressure	UV Index	Season	Visibility (km)	Location	Weather Type
0	14.0	73	9.5	82.0	partly cloudy	1010.82	2	Winter	3.5	inland	Rainy
1	39.0	96	8.5	71.0	partly cloudy	1011.43	7	Spring	10.0	inland	Cloudy
2	30.0	64	7.0	16.0	clear	1018.72	5	Spring	5.5	mountain	Sunny
3	38.0	83	1.5	82.0	clear	1026.25	7	Spring	1.0	coastal	Sunny
4	27.0	74	17.0	66.0	overcast	990.67	1	Winter	2.5	mountain	Rainy

Kode memakai `pd.read_csv(...)` untuk memuat dataset cuaca dari Google Drive ke dalam DataFrame `df`, lalu dilanjutkan dengan `df.head()` untuk melihat beberapa baris pertama. Input-nya adalah path file CSV dan perintah menampilkan head dari DataFrame. Output-nya berupa DataFrame yang berisi seluruh data cuaca, ditambah tabel kecil 5 baris awal sebagai preview agar kita memahami struktur kolom dan contoh isinya.

- Mengecek Informasi Struktur Dataset (`df.info()`)

```
[19] df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 13200 entries, 0 to 13199
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   Temperature            13200 non-null  float64
1   Humidity                13200 non-null  int64
2   Wind Speed              13200 non-null  float64
3   Precipitation (%)       13200 non-null  float64
4   Cloud Cover             13200 non-null  object
5   Atmospheric Pressure     13200 non-null  float64
6   UV Index                13200 non-null  int64
7   Season                  13200 non-null  object
8   Visibility (km)         13200 non-null  float64
9   Location                13200 non-null  object
10  Weather Type            13200 non-null  object
dtypes: float64(5), int64(2), object(4)
memory usage: 1.1+ MB
```

Kode `df.info()` dipakai untuk melihat jumlah baris, kolom, serta tipe data tiap kolom. Input-nya adalah DataFrame `df`. Output-nya ringkasan struktur dataset, termasuk total 13.200 baris dan tipe data fitur cuaca.

- Mengecek Missing Value

```
[20]
✓ 0s
df.isnull().sum()

0
Temperature      0
Humidity          0
Wind Speed        0
Precipitation (%) 0
Cloud Cover       0
Atmospheric Pressure 0
UV Index          0
Season            0
Visibility (km)    0
Location          0
Weather Type      0
dtype: int64
```

Kode `df.isnull().sum()` menghitung apakah ada nilai kosong di setiap kolom. Input-nya DataFrame `df`. Output-nya jumlah missing value per kolom, dan hasilnya menunjukkan semua kolom bernilai 0 (tidak ada data kosong).

- Memisahkan Fitur (X) dan Target (y)

```
[21]
✓ 0s
X = df.drop(columns=["Weather Type"])
y = df["Weather Type"]
```

Kode membuat X dari semua kolom kecuali “Weather Type” dan y dari kolom “Weather Type”. Input-nya adalah DataFrame cuaca. Output-nya dua variabel: X sebagai data input model dan y sebagai label tipe cuaca yang akan diprediksi.

- Menentukan Kolom Kategorikal dan Numerik

```
[22]
✓ 0s
categorical_cols = ["Cloud Cover", "Season", "Location"]
numeric_cols = [col for col in X.columns if col not in categorical_cols]

numeric_cols, categorical_cols

(['Temperature',
 'Humidity',
 'Wind Speed',
 'Precipitation (%)',
 'Atmospheric Pressure',
 'UV Index',
 'Visibility (km)'],
 ['Cloud Cover', 'Season', 'Location'])
```

Kode ini dipakai untuk memisahkan fitur mana yang termasuk data kategorikal dan mana yang numerik. Input-nya adalah daftar kolom kategorikal yang ditulis manual, yaitu `["Cloud Cover", "Season", "Location"]`, lalu semua kolom lain di

X.columns otomatis dimasukkan ke numeric_cols lewat list comprehension. Output-nya menampilkan dua list: numeric_cols berisi kolom angka seperti Temperature, Humidity, Wind Speed, dan seterusnya, sedangkan categorical_cols tetap berisi tiga kolom kategori tadi. Pemisahan ini penting supaya preprocessing bisa berbeda: numerik distandardisasi, kategorikal di-one-hot encoding.

- Membuat Preprocessor dengan ColumnTransformer

```
[23]
✓ 0s
preprocessor = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), numeric_cols),
        ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_cols)
    ]
)
```

Kode ini membuat objek preprocessor untuk menyiapkan data sebelum masuk ke model KNN. Input-nya adalah dua jenis transformer: bagian "num" memakai StandardScaler() untuk kolom numerik (numeric_cols) supaya semua fitur angka punya skala yang seragam, dan bagian "cat" memakai OneHotEncoder(handle_unknown="ignore") untuk kolom kategorikal (categorical_cols) agar nilai kategori diubah menjadi bentuk angka biner. Output-nya adalah sebuah ColumnTransformer bernama preprocessor yang berisi aturan preprocessing lengkap, sehingga nanti saat pipeline dijalankan, data otomatis distandardisasi dan di-encode sesuai jenis kolomnya.

- Membagi Data Train dan Test

```
[24]
✓ 0s
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,          # 20% test
    random_state=42,
    stratify=y              # biar proporsi kelas seimbang
)

print("Train size:", X_train.shape)
print("Test size :", X_test.shape)

Train size: (10560, 10)
Test size : (2640, 10)
```

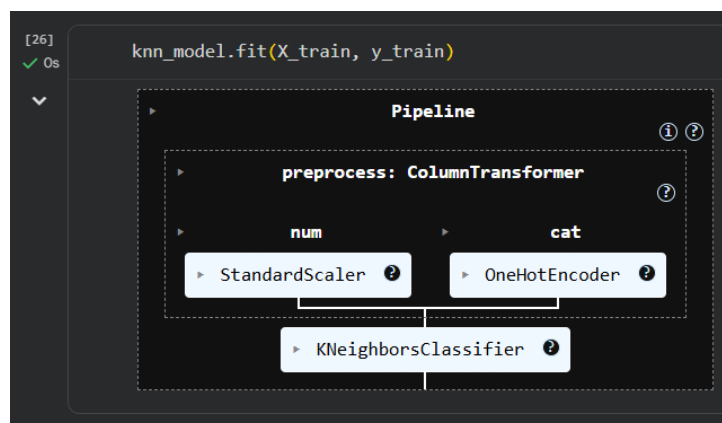
Kode train_test_split(X, y, test_size=0.2, stratify=y, random_state=42) membagi data. Input-nya fitur X dan target y. Output-nya X_train, X_test, y_train, y_test dengan ukuran train 10.560 dan test 2.640, serta proporsi kelas tetap seimbang.

- Membuat Pipeline Preprocessing dan Model KNN

```
[25] knn_model = Pipeline(steps=[  
    ✓ Os      ("preprocess", preprocessor),  
            ("knn", KNeighborsClassifier(n_neighbors=5)) # default awal K=5  
        ])
```

Kode ini menyusun sebuah Pipeline bernama `knn_model` yang menggabungkan dua langkah berurutan. Input-nya adalah daftar steps yang berisi: langkah pertama "preprocess" memakai objek preprocessor untuk melakukan standardisasi data numerik dan one-hot encoding data kategorikal, lalu langkah kedua "knn" memakai `KNeighborsClassifier(n_neighbors=5)` sebagai model klasifikasi dengan nilai K awal = 5. Output-nya adalah pipeline utuh yang siap dilatih dan dipakai prediksi, sehingga saat fit atau predict dipanggil, data otomatis diproses dulu baru dihitung oleh KNN.

- Melatih Model dengan Data Train



Kode menjalankan `knn_model.fit(X_train, y_train)`. Input-nya data train. Output-nya model pipeline yang sudah belajar pola hubungan fitur cuaca dengan label tipe cuaca.

- Memprediksi Data Test

```
[27] y_pred = knn_model.predict(X_test)  
     y_pred[:10]  
     array(['Rainy', 'Cloudy', 'Sunny', 'Rainy', 'Cloudy', 'Cloudy', 'Cloudy',  
            'Rainy', 'Cloudy', 'Sunny'], dtype=object)
```

Kode `knn_model.predict(X_test)` menghasilkan prediksi untuk data uji. Input-nya fitur `X_test`. Output-nya `y_pred`, yaitu label tipe cuaca hasil prediksi model.

- Menghitung Akurasi Model

```
[28]
✓ 0s
acc = accuracy_score(y_test, y_pred)
print("Accuracy:", acc)
print("Accuracy (%)ate", acc*100, "%")

Accuracy: 0.8962121212121212
Accuracy (%): 89.62121212121212 %
```

Kode `accuracy_score(y_test, y_pred)` menghitung ketepatan prediksi keseluruhan. Input-nya label aktual dan label prediksi. Output-nya nilai akurasi sekitar 0.8962 (89.62%).

- Menampilkan Classification Report

```
[29]
✓ 0s
print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
Cloudy	0.84	0.90	0.87	660
Rainy	0.87	0.90	0.89	660
Snowy	0.93	0.91	0.92	660
Sunny	0.95	0.88	0.91	660
accuracy			0.90	2640
macro avg	0.90	0.90	0.90	2640
weighted avg	0.90	0.90	0.90	2640

Kode `classification_report(y_test, y_pred)` menampilkan evaluasi per kelas cuaca. Input-nya label aktual dan prediksi. Output-nya tabel precision, recall, f1-score, dan support untuk tiap tipe cuaca.

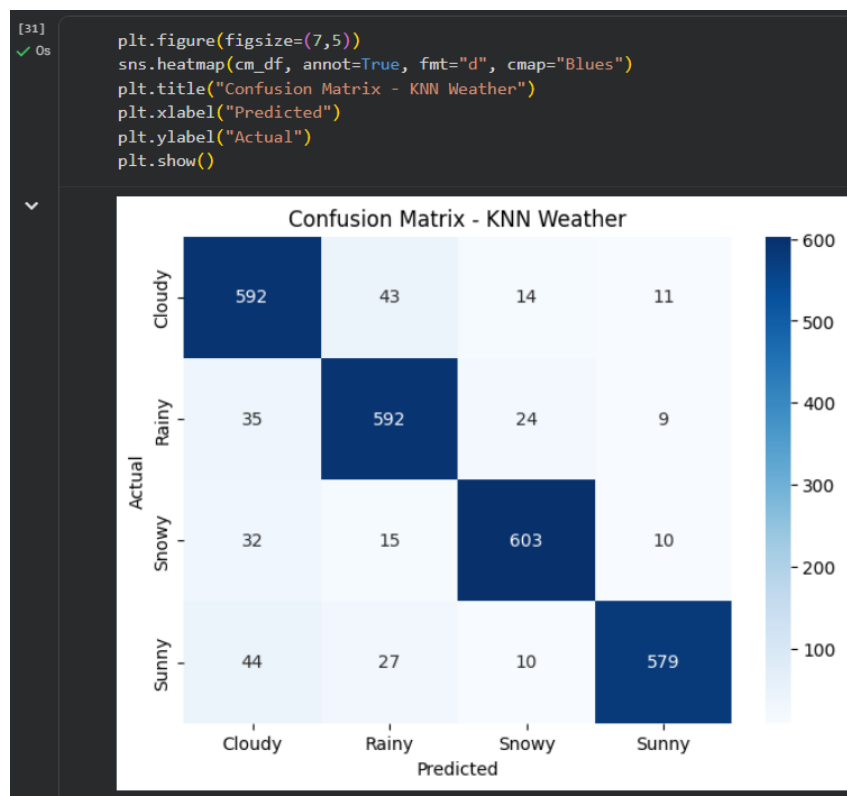
- Membuat Confusion Matrix

```
[30]
✓ Os
cm = confusion_matrix(y_test, y_pred, labels=knn_model.classes_)
cm_df = pd.DataFrame(cm, index=knn_model.classes_, columns=knn_model.classes_)
cm_df
```

	Cloudy	Rainy	Snowy	Sunny
Cloudy	592	43	14	11
Rainy	35	592	24	9
Snowy	32	15	603	10
Sunny	44	27	10	579

Kode `confusion_matrix(y_test, y_pred)` menghitung jumlah prediksi benar/salah per kelas. Input-nya label aktual dan prediksi. Output-nya matriks evaluasi yang jadi dasar analisis error model.

- Visualisasi Confusion Matrix dengan Heatmap



Kode memakai seaborn heatmap untuk menampilkan confusion matrix agar mudah dibaca. Input-nya confusion matrix. Output-nya gambar heatmap yang menunjukkan area prediksi benar (diagonal tinggi) dan salah (di luar diagonal).

- Kesimpulan

Setelah mengerjakan praktikum mandiri, dapat disimpulkan bahwa proses preprocessing seperti pemisahan fitur numerik dan kategorikal, standardisasi, dan one-hot encoding sangat membantu menyiapkan data agar cocok untuk KNN. Model KNeighborsClassifier yang dibuat dalam pipeline dapat melakukan klasifikasi tipe cuaca dengan performa yang baik, terlihat dari nilai accuracy yang tinggi serta hasil classification report dan confusion matrix yang menunjukkan prediksi model cukup tepat pada sebagian besar kelas. Secara keseluruhan, KNN efektif digunakan untuk klasifikasi cuaca jika data diolah dengan langkah yang benar.

Link Praktikum 10:

https://github.com/HisyamWildan/TI03_HisyamW.A_0110222206/blob/main/Pertemuan%2010/Praktikum%20dan%20Tugas%20Mandiri%2010/notebooks/praktikum10.ipynb

Tugas Mandiri 10:

https://github.com/HisyamWildan/TI03_HisyamW.A_0110222206/blob/main/Pertemuan%2010/Praktikum%20dan%20Tugas%20Mandiri%2010/notebooks/latihan10.ipynb